Creating your Blog Application

- The blog application will consist of a list of posts including the post title, publishing date, author, a post excerpt, and a link to read the post.
- The post list page will be implemented with the post_list view. You will learn how to create views in this chapter.
- Command 1:

  ```
  django-admin startproject mysite
  ```

The outer mysite/ directory is the container for our project. It contains the following files:

- manage.py : This is a command-line utility used to interact with your project. You won't usually need to edit this file.
- mysite/ : This is the Python package for your project, which consists of the following files:
- __init__.py : An empty file that tells Python to treat the mysite directory as a Python module.
- asgi.py : This is the configuration to run your project as an ASGI application with ASGI-compatible web servers. ASGI is the emerging Python standard for asynchronous web servers and applications.
- settings.py : This indicates settings and configuration for your project and contains initial default settings.

- urls.py : This is the place where your URL patterns live. Each URL defined here is mapped to a view.
- wsgi.py : This is the configuration to run your project as a **Web Server Gateway Interface** (**WSGI**) application with WSGIcompatible web servers.

ASGI and WSGI

- Django is a Python framework. But browsers don't talk to Python directly. They communicate using **HTTP**. So we need a **bridge** between: Web Server ↔ Django (Python)
- That bridge is:

  WSGI -Web Server Gateway Interface (older, synchronous)

  ASGI -Asynchronous Server Gateway Interface (newer, asynchronous)
- WSGI handles:

  HTTP request

  Converts it into Python objects

  Sends response back
- **WSGI is Synchronous** meaning one request at a time per worker

- ASGI is the modern version of WSGI. It supports Async views.
- WSGI and ASGI are interfaces that allow web servers to communicate with Django, where WSGI is synchronous and ASGI supports asynchronous communication.
- **Command 2**

  cd mysite

  python manage.py migrate
- The preceding lines are the database migrations that are applied by Django. By applying the initial migrations, the tables for the applications listed in the INSTALLED_APPS setting are created in the database.

- **Command 3**

  python manage.py runserver

- This server is only intended for development and is not suitable for production use. To deploy Django in a production environment, you should run it as a WSGI application using a web server, such as Apache, Gunicorn, or uWSGI, or as an ASGI application using a server such as Daphne or Uvicorn.
- **Command 4 (from the Project's root directory)**

  python manage.py startapp blog

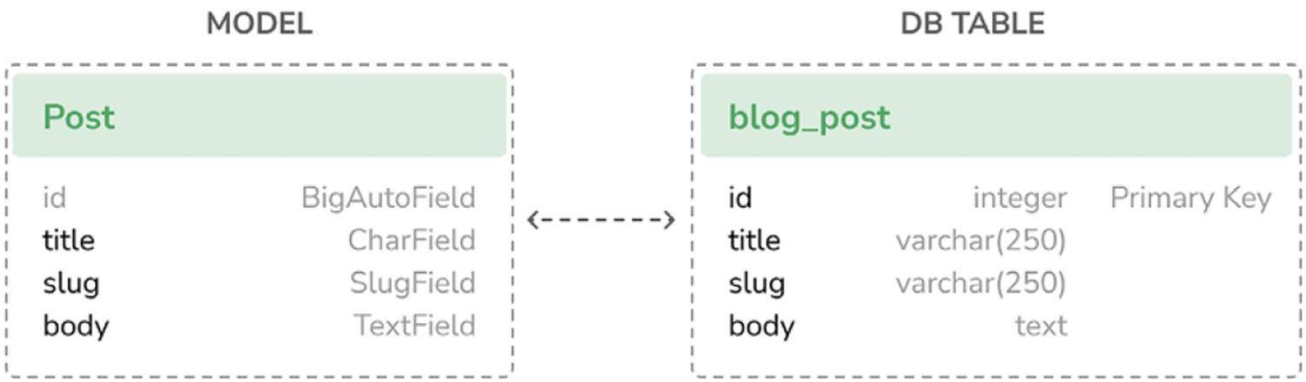
Creating the blog data models

- **Creating the Post model**
- First, we will define a Post model that will allow us to store blog posts in the database.

```
from django.db import models
class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    def __str__(self):
        return self.title
```

- title : This is the field for the post title. This is a CharField field that translates into a VARCHAR column in the SQL database.
- slug : This is a SlugField field that translates into a VARCHAR column in the SQL database. A slug is a short label that contains only letters, numbers, underscores, or hyphens. A post with the title *Django Reinhardt: A legend of Jazz* could have a slug like *django-reinhardtlegend-jazz*.
- body : This is the field for storing the body of the post. This is a TextField field that translates into a TEXT column in the SQL database.
- We have also added a __str__() method to the model class. This is the default Python method to return a string with the human-readable representation of the object.

Post Model to database table



Adding datetime fields

- **publish = models.DateTimeField(default=timezone.now)**
- **created = models.DateTimeField(auto_now_add=True)**
- **updated = models.DateTimeField(auto_now=True)**
- We have added a publish field to the Post model. This is a DateTimeField field that translates into a DATETIME column in the SQL database. We will use it to store the date and time when the post was published. We use Django's timezone.now method as the default value for the field.

We have added the following fields to the Post model:

- created : This is a DateTimeField field. We will use it to store the date and time when the post was created. By using auto_now_add , the date will be saved automatically when creating an object.
- updated : This is a DateTimeField field. We will use it to store the last date and time when the post was updated. By using auto_now , the date will be updated automatically when saving an object.

Defining a default sort order

- Blog posts are typically presented in reverse chronological order, showing the newest posts first.
- For our model, we will define a default ordering. This ordering takes effect when retrieving objects from the database unless a specific order is indicated in the query.

class Meta:

```
    ordering = ['-publish']
```

- We use the ordering attribute to tell Django that it should sort results by the publish field.
- We indicate descending order by using a hyphen before the field name, - publish .

Adding a database index

- Define a database index for the publish field. This will improve performance for query filtering or ordering results by this field.

```
class Meta:
    ordering = ['-publish']
    indexes = [
        models.Index(fields=['-publish']),
    ]
```

- We use a hyphen before the field name to define the index specifically in descending order.

Activating the application

```
INSTALLED_APPS = [
'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'blog.apps.BlogConfig',
]
```

Adding a status field

- A common functionality for blogs is to save posts as a draft until ready for publication.
- We will add a status field to our model that will allow us to manage the status of blog posts. We will be using the *Draft* and *Published* statuses for posts.

```
class Status(models.TextChoices):
    DRAFT = 'DF', 'Draft'
    PUBLISHED = 'PB', 'Published'
status = models.CharField(
max_length=2,
choices=Status,
default=Status.DRAFT
)
```

We can access :

- Post.Status.choices to obtain the available choices,
- Post.Status.names to obtain the names of the choices,
- Post.Status.labels to obtain the human-readable names, and
- Post.Status.values to obtain the actual values of the choices.
- **Command 5**

```
python manage.py shell
>>> from blog.models import Post
>>> Post.Status.choices
>>> Post.Status.labels
>>> Post.Status.values
>>> Post.Status.names
```

Adding a many-to-one relationship

- Posts are always written by an author. We will create a relationship between users and posts that will indicate which user wrote which posts. Django comes with an authentication framework that handles user accounts.

```
author = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.CASCADE,
    related_name='blog_posts'
)
```

Creating and applying migrations

- python manage.py makemigrations blog
- Run the following command from the shell prompt to inspect the SQL output of your first migration:
  - python manage.py sqlmigrate blog 0001
- Execute the following command in the shell prompt to apply the existing migrations:
  - python manage.py migrate

Creating a superuser

- **Command**

  python manage.py createsuperuser
- Username (leave blank to use 'admin'): admin
- Email address: admin@admin.com
- Password: ********
- Password (again): ********
- Now, run the server using **python manage.py runserver**

- Open http://127.0.0.1:8000/admin/ in your browser
- The Group and User models that you can see are part of the Django authentication framework located in **django.contrib.auth**. If you click on **Users**, you will see the user you created previously.
- Edit the admin.py file of the blog application

from django.contrib import admin

from .models import Post

admin.site.register(Post)

- Now, reload the admin site on the browser to see the Post model.

Customizing how models are displayed

- Edit the admin.py file of your blog application and change it, as follows.

```
from django.contrib import admin
from .models import Post
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
        list_display = ['title', 'slug', 'author', 'publish', 'status’]
```

- We are telling the Django administration site that the model is registered in the site using a custom class that inherits from ModelAdmin.
- In this class, we can include information about how to display the model on the administration site and how to interact with it.

- The list_display attribute allows you to set the fields of your model that you want to display on the administration object list page. The
- @admin.register() decorator performs the same function as the admin.site.register() function that you replaced, registering the ModelAdmin class that it decorates.

- Edit the admin.py file of your blog application and change it, as follows.

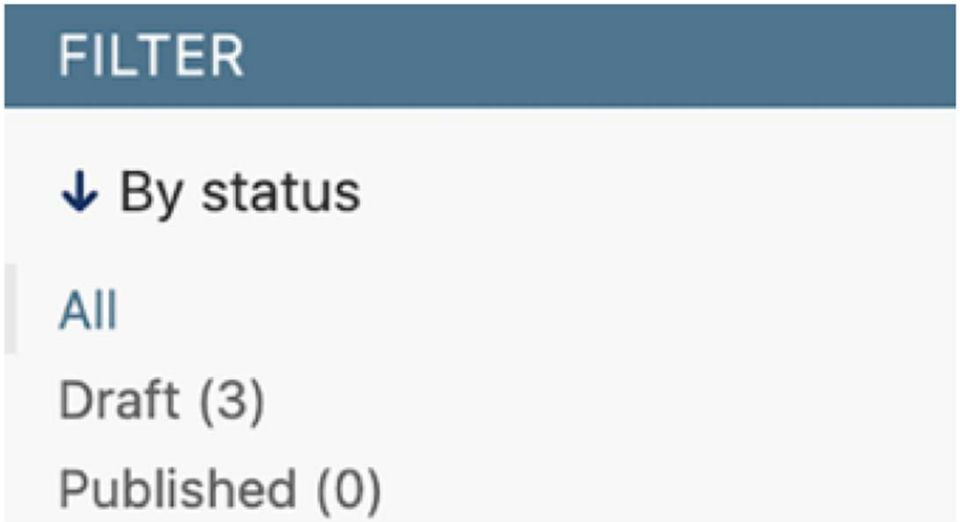
```
from django.contrib import admin
from .models import Post
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug', 'author', 'publish', 'status’]
    list_filter = ['status', 'created', 'publish', 'author']
    search_fields = ['title', 'body']
    prepopulated_fields = {'slug': ('title',)}
    raw_id_fields = ['author']
    date_hierarchy = 'publish'
    ordering = ['status', 'publish']
```

Adding facet counts to filters

- Django 5.0 introduces facet filters to the administration site, showcasing facet counts. These counts indicate the number of objects corresponding to each specific filter, making it easier to identify matching objects .
- Facet counts, often called "smart filters" or faceted navigation, are UI elements that display the number of items matching specific attributes (e.g., Brand: Nike (15), Adidas (10)) within a search result. These counts allow users to narrow down large datasets by filtering results based on categories like brand, color, or price.

- Edit the admin.py file of your blog application and add the following

  show_facets = admin.ShowFacets.ALWAYS



Working with QuerySets and managers

- To read and write content to the database programmatically.
- The Django **object-relational mapper** (**ORM**) is a powerful database abstraction API that lets you create, retrieve, update, and delete objects easily. An ORM allows you to generate SQL queries using the object oriented paradigm
- The ORM maps your models to database tables and provides you with a simple Pythonic interface to interact with your database. of Python.
- The Django ORM is compatible with MySQL, PostgreSQL, SQLite, Oracle, and MariaDB.

Working with QuerySets and managers

- The Django ORM is based on QuerySets. A QuerySet is a collection of database queries to retrieve objects from your database.
- We can apply filters to QuerySets to narrow down the query results based on given parameters. The QuerySet equates to a SELECT SQL statement and the filters are limiting SQL clauses such as WHERE or LIMIT .

Creating objects

- python manage.py shell

```
>>> from django.contrib.auth.models import User
>>> from blog.models import Post
>>> user = User.objects.get(username='admin')
>>> post = Post(title='Post on February 27', slug='February-post',
body='Post body on february 27.', author=user)
>>> post.save()
```

Updating objects

- Now, change the title of the previous Post object to something different and save the object again:

```
>>> post.title = 'Neww title'
>>> post.save()
```

- This time, the save() method performs an UPDATE SQL statement.
- The changes we make to a model object are not persisted to the database until you call the save() method.

Retrieving objects

- To retrieve a single object from the database use the get() method.

```
Post.objects.get()
```

- Each Django model has at least one manager, and the default manager is called objects.
- To retrieve all objects from a table, we use the all() method on the default objects manager, like this:

```
all_posts = Post.objects.all()
```

- This is how we create a QuerySet that returns all objects in the database.

Retrieving objects

- If you don’t assign the QuerySet to a variable but, instead, write it directly on the Python shell, the SQL statement of the QuerySet is executed because you are forcing it to generate output:

```
Post.objects.all()
```

Filtering objects

- To filter a QuerySet, you can use the filter() method of the manager. This method allows you to specify the content of a SQL WHERE clause.
- For example, you can use the following to filter Post objects by their title :

```
Post.objects.filter(title='Who was Django Reinhardt?')
```

- This QuerySet will return all posts with the exact title *Who was Django Reinhardt?*.

Filtering objects

- The SQL statement generated with this QuerySet.

```
posts = Post.objects.filter(title='Who was Django Reinhardt?')
print(posts.query)
```

- There are multiple ways to retrieve objects:

```
Post.objects.filter(publish__month=2)
Post.objects.filter(publish__year=2024)
Post.objects.filter(publish__month=1)
Post.objects.filter(publish__day=1)
```

Creating model managers

- The default manager for every model is the objects manager. This manager retrieves all the objects in the database. However, we can define custom managers for models.
- Create a custom manager to retrieve all posts that have a PUBLISHED status.

```
class PublishedManager(models.Manager):
    def get_queryset(self):
        return (
        super().get_queryset().filter(status=Post.Status.PUBLISHED)
        )
```

Creating model managers

- objects = models.Manager() *# The default manager.*
- published = PublishedManager() *# Our custom manager.*
- If no manager is defined in the model, Django automatically creates the objects default manager for it.
- The get_queryset() method of a manager returns the QuerySet that will be executed.
- We have overridden this method to build a custom QuerySet that filters posts by their status and returns a successive QuerySet that only includes posts with the PUBLISHED status.

- We have now defined a custom manager for the Post model.
- python manage.py s

```
>>> from blog.models import Post
>>> Post.published.filter(title__startswith='Who')
```

Creating list and detail views

```
from django.shortcuts import render
from .models import Post
def post_list(request):
    posts = Post.published.all()
    return render(
        request,
        'blog/post/list.html',
        {'posts': posts}
    )
```

Creating list and detail views

```
def post_detail(request, id):
    try:
        post = Post.published.get(id=id)
    except Post.DoesNotExist:
        raise Http404("No Post found.")
    return render(
        request,
        'blog/post/detail.html',
        {'post': post}
    )
```

Creating list and detail views

```
def post_detail(request, id):
    post = get_object_or_404(
    Post,
    id=id,
    status=Post.Status.PUBLISHED
    )
    return render(
        request,
        'blog/post/detail.html',
        {'post': post}
    )
```

Adding URL patterns for your views

- Create a urls.py file in the directory of the blog application and add the following lines to it:

```
from django.urls import path
from . import views
app_name = 'blog'
urlpatterns = [
    # post views
    path('', views.post_list, name='post_list'),
    path('<int:id>/', views.post_detail, name='post_detail'),
]
```

- Edit the urls.py file located in the mysite directory of your project:

```
from django.contrib import admin
from django.urls import include, path
urlpatterns = [
path('admin/', admin.site.urls),
path('blog/', include('blog.urls', namespace='blog')),
]
```

Creating templates for your views

- Create the following directories and files inside your blog application directory:

```
templates/
    blog/
        base.html
        post/
            list.html
            detail.html
```

Creating a base template

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
<title>{% block title %}{% endblock %}</title>
<link href="{% static "css/blog.css" %}" rel="stylesheet">
</head>
<body>
<div id="content">
{% block content %}
{% endblock %}
</div>
<div id="sidebar">
<h2>My blog</h2>
<p>This is my blog.</p>
</div>
</body>
</html>
```

Creating a base template

- {% load static %} tells Django to load the static template tags that are provided by the django.contrib.staticfiles application, which is contained in the INSTALLED_APPS setting. After loading them, you can use the {% static %} template tag throughout this template. With this template tag, you can include the static files, such as the blog.css file.

Creating the post list template

```
{% extends "blog/base.html" %}
{% block title %}My Blog{% endblock %}
{% block content %}
<h1>My Blog</h1>
{% for post in posts %}
<h2>
<a href="{% url 'blog:post_detail' post.id %}">
{{ post.title }}
</a>
</h2>
<p class="date">
Published {{ post.publish }} by {{ post.author }}
</p>
{{ post.body|truncatewords:30|linebreaks }}
{% endfor %}
{% endblock %}
```

Creating the post detail template

```
{% extends "blog/base.html" %}
{% block title %}{{ post.title }}{% endblock %}
{% block content %}
<h1>{{ post.title }}</h1>
<p class="date">
Published {{ post.publish }} by {{ post.author }}
</p>
{{ post.body|linebreaks }}
{% endblock %}
```